Comprehensive Guide to Modern System Development: Principles, Patterns & Practices

---

1. Foundations of System Development

1.1 Evolution of System Architecture

Historical Context:
- **1960s-70s**: Monolithic Mainframe Systems
- **1980s-90s**: Client-Server Architecture
- **2000s**: Service-Oriented Architecture (SOA)
- **2010s**: Microservices & Cloud-Native
- **2020s**: Serverless, Edge Computing & AI-Driven Systems

1.2 Core System Development Principles

SOLID Principles:
```java
// Single Responsibility Principle
class UserService {
    // Only handles user-related operations
    public User createUser(UserData data) { ... }
    public User getUserById(String id) { ... }
}

class EmailService {
    // Separate service for email operations
    public void sendWelcomeEmail(User user) { ... }
```

```
}

// Open/Closed Principle
interface PaymentProcessor {
    void processPayment(PaymentData data);
}

class StripeProcessor implements PaymentProcessor {
    public void processPayment(PaymentData data) {
        // Stripe-specific implementation
    }
}

class PayPalProcessor implements PaymentProcessor {
    public void processPayment(PaymentData data) {
        // PayPal-specific implementation
    }
}
```

DRY (Don't Repeat Yourself) & KISS (Keep It Simple, Stupid):

```python
# BEFORE: Repetitive code
def calculate_order_total(order):
    subtotal = sum(item.price * item.quantity for item in order.items)
    tax = subtotal * 0.08
    shipping = 5.99 if subtotal < 50 else 0
    return subtotal + tax + shipping
```

```
def calculate_invoice_total(invoice):
    subtotal = sum(item.price * item.quantity for item in invoice.items)
    tax = subtotal * 0.08
    shipping = 5.99 if subtotal < 50 else 0
    return subtotal + tax + shipping

# AFTER: DRY implementation
class PriceCalculator:
    TAX_RATE = 0.08
    FREE_SHIPPING_THRESHOLD = 50
    SHIPPING_COST = 5.99

    def calculate_total(self, items):
        subtotal = self._calculate_subtotal(items)
        tax = self._calculate_tax(subtotal)
        shipping = self._calculate_shipping(subtotal)
        return subtotal + tax + shipping

    def _calculate_subtotal(self, items):
        return sum(item.price * item.quantity for item in items)

    def _calculate_tax(self, subtotal):
        return subtotal * self.TAX_RATE

    def _calculate_shipping(self, subtotal):
        return 0 if subtotal >= self.FREE_SHIPPING_THRESHOLD else self.SHIPPING_COST
```

---

2. Modern System Architecture Patterns

2.1 Microservices Architecture

Core Concepts:

```yaml
# docker-compose.yml - Microservices setup
version: '3.8'
services:
  user-service:
    build: ./services/user
    ports:
      - "8001:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@user-db:5432/users
      - RABBITMQ_URL=amqp://rabbitmq:5672
    depends_on:
      - user-db
      - rabbitmq

  order-service:
    build: ./services/order
    ports:
      - "8002:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@order-db:5432/orders
      - RABBITMQ_URL=amqp://rabbitmq:5672
```

```
  api-gateway:
    build: ./gateway
    ports:
      - "8000:8000"
    environment:
      - USER_SERVICE_URL=http://user-service:8000
      - ORDER_SERVICE_URL=http://order-service:8000
```

Service Communication:

```python
# services/user/service.py
from flask import Flask, jsonify
import requests
import pika
import json

app = Flask(__name__)

# Synchronous communication (HTTP)
@app.route('/users/<user_id>/orders')
def get_user_orders(user_id):
    user = get_user(user_id)
    orders_response = requests.get(f'http://order-service:8000/orders?user_id={user_id}')
    orders = orders_response.json()

    return jsonify({
        'user': user,
        'orders': orders
```

```
    })

# Asynchronous communication (Message Queue)
def publish_user_created_event(user_data):
    connection = pika.BlockingConnection(
        pika.ConnectionParameters(host='rabbitmq')
    )
    channel = connection.channel()

    channel.queue_declare(queue='user_events')

    channel.basic_publish(
        exchange='',
        routing_key='user_events',
        body=json.dumps({
            'event_type': 'USER_CREATED',
            'user_id': user_data['id'],
            'email': user_data['email']
        })
    )
    connection.close()
```

2.2 Event-Driven Architecture

Event Sourcing Pattern:
```python
# event_store.py
class EventStore:
```

```
    def __init__(self):
        self.events = []

    def append_event(self, aggregate_id, event_type, event_data):
        event = {
            'event_id': len(self.events) + 1,
            'aggregate_id': aggregate_id,
            'event_type': event_type,
            'event_data': event_data,
            'timestamp': datetime.utcnow().isoformat()
        }
        self.events.append(event)
        return event

    def get_events_for_aggregate(self, aggregate_id):
        return [e for e in self.events if e['aggregate_id'] == aggregate_id]

# order_aggregate.py
class OrderAggregate:
    def __init__(self, event_store):
        self.event_store = event_store
        self.state = {
            'order_id': None,
            'status': 'PENDING',
            'items': [],
            'total_amount': 0
        }

    def create_order(self, order_data):
```

```
        event = self.event_store.append_event(
            order_data['order_id'],
            'ORDER_CREATED',
            order_data
        )
        self.apply_event(event)

    def apply_event(self, event):
        if event['event_type'] == 'ORDER_CREATED':
            self.state.update(event['event_data'])
            self.state['status'] = 'CREATED'

        elif event['event_type'] == 'ORDER_PAID':
            self.state['status'] = 'PAID'
            self.state['payment_id'] = event['event_data']['payment_id']
```

CQRS (Command Query Responsibility Segregation):

```python
# Command Side (Write Model)
class OrderCommandHandler:
    def __init__(self, event_store, command_bus):
        self.event_store = event_store
        self.command_bus = command_bus

    def handle_create_order(self, command):
        # Validate command
        if not command.is_valid():
            raise ValidationError("Invalid command")
```

```
        # Create event
        event = OrderCreatedEvent(
            order_id=command.order_id,
            user_id=command.user_id,
            items=command.items
        )

        # Store event
        self.event_store.append_event(event)

        # Publish to query side
        self.command_bus.publish(event)

# Query Side (Read Model)
class OrderQueryService:
    def __init__(self, database):
        self.db = database

    def get_order_details(self, order_id):
        # Read from optimized read database
        return self.db.orders.find_one({'_id': order_id})

    def get_user_orders(self, user_id):
        return self.db.orders.find({'user_id': user_id})
```

---

3. System Design Patterns

3.1 Circuit Breaker Pattern

```python
# circuit_breaker.py
import time
from enum import Enum
from typing import Callable, Any

class CircuitState(Enum):
    CLOSED = "CLOSED"
    OPEN = "OPEN"
    HALF_OPEN = "HALF_OPEN"

class CircuitBreaker:
    def __init__(
        self,
        failure_threshold: int = 5,
        recovery_timeout: int = 60,
        expected_exception: type = Exception
    ):
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.expected_exception = expected_exception

        self.state = CircuitState.CLOSED
        self.failure_count = 0
        self.last_failure_time = None
```

```
def call(self, func: Callable, *args, **kwargs) -> Any:
    if self.state == CircuitState.OPEN:
        if self._is_recovery_timeout_elapsed():
            self.state = CircuitState.HALF_OPEN
        else:
            raise CircuitBreakerOpenException("Circuit breaker is OPEN")

    try:
        result = func(*args, **kwargs)
        self._on_success()
        return result

    except self.expected_exception as e:
        self._on_failure()
        raise

def _on_success(self):
    self.failure_count = 0
    self.last_failure_time = None
    self.state = CircuitState.CLOSED

def _on_failure(self):
    self.failure_count += 1
    self.last_failure_time = time.time()

    if self.failure_count >= self.failure_threshold:
        self.state = CircuitState.OPEN
```

```
    def _is_recovery_timeout_elapsed(self):
        if not self.last_failure_time:
            return True

        return (time.time() - self.last_failure_time) > self.recovery_timeout

# Usage
breaker = CircuitBreaker(failure_threshold=3, recovery_timeout=30)

try:
    result = breaker.call(external_api.call, api_params)
    return result
except CircuitBreakerOpenException:
    # Return cached response or default value
    return get_cached_response()
```

3.2 Repository Pattern

```python
# repository.py
from abc import ABC, abstractmethod
from typing import List, Optional, TypeVar, Generic

T = TypeVar('T')
ID = TypeVar('ID')

class Repository(Generic[T, ID], ABC):
    @abstractmethod
```

```
    def get_by_id(self, id: ID) -> Optional[T]:
        pass

    @abstractmethod
    def get_all(self) -> List[T]:
        pass

    @abstractmethod
    def add(self, entity: T) -> T:
        pass

    @abstractmethod
    def update(self, entity: T) -> T:
        pass

    @abstractmethod
    def delete(self, id: ID) -> bool:
        pass

# Concrete implementation
class UserRepository(Repository[User, str]):
    def __init__(self, session_factory):
        self.session_factory = session_factory

    def get_by_id(self, user_id: str) -> Optional[User]:
        with self.session_factory() as session:
            return session.query(User).filter(User.id == user_id).first()

    def get_by_email(self, email: str) -> Optional[User]:
```

```
        with self.session_factory() as session:
            return session.query(User).filter(User.email == email).first()

    def add(self, user: User) -> User:
        with self.session_factory() as session:
            session.add(user)
            session.commit()
            return user

    def update(self, user: User) -> User:
        with self.session_factory() as session:
            session.merge(user)
            session.commit()
            return user
```

3.3 Strategy Pattern

```python
# payment_strategies.py
from abc import ABC, abstractmethod

class PaymentStrategy(ABC):
    @abstractmethod
    def process_payment(self, amount: float, payment_data: dict) -> dict:
        pass

class StripePaymentStrategy(PaymentStrategy):
    def process_payment(self, amount: float, payment_data: dict) -> dict:
```

```
        # Stripe-specific implementation
        import stripe
        stripe.api_key = "sk_test_..."

        try:
            payment_intent = stripe.PaymentIntent.create(
                amount=int(amount * 100),  # Convert to cents
                currency='usd',
                payment_method=payment_data['payment_method_id'],
                confirm=True
            )

            return {
                'success': True,
                'transaction_id': payment_intent.id,
                'status': payment_intent.status
            }
        except stripe.error.StripeError as e:
            return {
                'success': False,
                'error': str(e)
            }

class PayPalPaymentStrategy(PaymentStrategy):
    def process_payment(self, amount: float, payment_data: dict) -> dict:
        # PayPal-specific implementation
        # ... PayPal API calls
        return {'success': True, 'transaction_id': 'paypal_123'}
```

```
class PaymentProcessor:
    def __init__(self):
        self._strategies = {}

    def register_strategy(self, provider: str, strategy: PaymentStrategy):
        self._strategies[provider] = strategy

    def process_payment(self, provider: str, amount: float, payment_data: dict) -> dict:
        strategy = self._strategies.get(provider)
        if not strategy:
            raise ValueError(f"Unknown payment provider: {provider}")

        return strategy.process_payment(amount, payment_data)

# Usage
processor = PaymentProcessor()
processor.register_strategy('stripe', StripePaymentStrategy())
processor.register_strategy('paypal', PayPalPaymentStrategy())

result = processor.process_payment(
    provider='stripe',
    amount=99.99,
    payment_data={'payment_method_id': 'pm_12345'}
)
```

---

4. Database Design & Optimization

4.1 Database Schema Design

Normalized Schema Example:

```sql
-- Users table
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Orders table
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    status VARCHAR(50) NOT NULL DEFAULT 'pending',
    total_amount DECIMAL(10,2) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Order items table
CREATE TABLE order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID REFERENCES orders(id),
```

```
    product_id UUID REFERENCES products(id),
    quantity INTEGER NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes for performance
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_users_email ON users(email);
```

4.2 Query Optimization

```sql
-- BEFORE: Inefficient query
SELECT *
FROM orders o
JOIN users u ON o.user_id = u.id
JOIN order_items oi ON o.id = oi.order_id
WHERE u.email = 'user@example.com'
AND o.created_at > NOW() - INTERVAL '30 days';

-- AFTER: Optimized query
EXPLAIN ANALYZE
SELECT
    o.id,
    o.status,
```

```
    o.total_amount,
    COUNT(oi.id) as item_count
FROM orders o
JOIN users u ON o.user_id = u.id
JOIN order_items oi ON o.id = oi.order_id
WHERE u.email = 'user@example.com'
AND o.created_at > NOW() - INTERVAL '30 days'
GROUP BY o.id, o.status, o.total_amount
ORDER BY o.created_at DESC;
```

4.3 Connection Pooling

```python
# database.py
from sqlalchemy import create_engine
from sqlalchemy.pool import QueuePool
import asyncpg
from contextlib import asynccontextmanager

# SQLAlchemy connection pool
engine = create_engine(
    'postgresql://user:pass@localhost/dbname',
    poolclass=QueuePool,
    pool_size=10,
    max_overflow=20,
    pool_timeout=30,
    pool_recycle=3600
)
```

```
# Async connection pool
class Database:
    def __init__(self):
        self.pool = None

    async def connect(self, dsn):
        self.pool = await asyncpg.create_pool(
            dsn,
            min_size=5,
            max_size=20,
            max_inactive_connection_lifetime=300
        )

    @asynccontextmanager
    async def acquire(self):
        async with self.pool.acquire() as connection:
            yield connection

    async def execute(self, query, *args):
        async with self.acquire() as conn:
            return await conn.execute(query, *args)
```

---

5. API Design & Development

5.1 RESTful API Best Practices

```python
# app/main.py
from fastapi import FastAPI, HTTPException, Depends, status
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI(
    title="E-Commerce API",
    description="Modern e-commerce platform API",
    version="1.0.0"
)

# Request/Response models
class ProductCreate(BaseModel):
    name: str
    description: str
    price: float
    stock_quantity: int

class ProductResponse(BaseModel):
    id: str
    name: str
    description: str
    price: float
    stock_quantity: int
    created_at: str

# API Routes
```

```
@app.post(
    "/products",
    response_model=ProductResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Create a new product",
    description="Create a new product in the catalog"
)
async def create_product(product: ProductCreate):
    try:
        # Business logic
        new_product = await product_service.create_product(product)
        return new_product
    except ValidationError as e:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=str(e)
        )

@app.get(
    "/products",
    response_model=List[ProductResponse],
    summary="List products",
    description="Get paginated list of products with filtering"
)
async def list_products(
    skip: int = 0,
    limit: int = 100,
    category: Optional[str] = None,
    min_price: Optional[float] = None,
```

```
    max_price: Optional[float] = None
):
    products = await product_service.get_products(
        skip=skip,
        limit=limit,
        category=category,
        min_price=min_price,
        max_price=max_price
    )
    return products

@app.get(
    "/products/{product_id}",
    response_model=ProductResponse,
    summary="Get product details",
    description="Get detailed information about a specific product"
)
async def get_product(product_id: str):
    product = await product_service.get_product_by_id(product_id)
    if not product:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Product not found"
        )
    return product
```

5.2 API Versioning

```python
# api/v1/router.py
from fastapi import APIRouter

router = APIRouter(prefix="/v1")

@router.get("/products")
async def get_products_v1():
    return {"version": "v1", "products": []}

# api/v2/router.py
router_v2 = APIRouter(prefix="/v2")

@router_v2.get("/products")
async def get_products_v2():
    return {
        "version": "v2",
        "products": [],
        "metadata": {
            "pagination": {...},
            "filters": {...}
        }
    }

# Main app
app.include_router(router, prefix="/api")
app.include_router(router_v2, prefix="/api")
```

---

6. Testing Strategies

6.1 Test Pyramid Implementation

```python
# tests/unit/test_services.py
import pytest
from unittest.mock import Mock, patch
from services.order_service import OrderService

class TestOrderService:
    def test_calculate_order_total(self):
        # Arrange
        service = OrderService()
        items = [
            {'price': 10.0, 'quantity': 2},
            {'price': 5.0, 'quantity': 1}
        ]

        # Act
        total = service.calculate_total(items)

        # Assert
        assert total == 25.0

    @patch('services.order_service.PaymentProcessor')
    def test_process_payment_success(self, mock_payment_processor):
```

```
        # Arrange
        mock_processor = Mock()
        mock_payment_processor.return_value = mock_processor
        mock_processor.charge.return_value = {'success': True, 'id': 'pay_123'}

        service = OrderService()

        # Act
        result = service.process_payment(100.0, 'pm_123')

        # Assert
        assert result['success'] == True
        mock_processor.charge.assert_called_once_with(100.0, 'pm_123')

# tests/integration/test_orders.py
class TestOrderIntegration:
    @pytest.mark.asyncio
    async def test_create_order_flow(self):
        # Create user
        user_response = await client.post("/api/users", json={
            "email": "test@example.com",
            "name": "Test User"
        })
        user_id = user_response.json()['id']

        # Create order
        order_response = await client.post("/api/orders", json={
            "user_id": user_id,
            "items": [{"product_id": "1", "quantity": 2}]
```

```
        })

        assert order_response.status_code == 201
        order_data = order_response.json()
        assert order_data['status'] == 'created'

# tests/e2e/test_checkout_flow.py
class TestCheckoutE2E:
    def test_complete_checkout_flow(self):
        # User visits site
        driver.get("https://store.example.com")

        # Adds product to cart
        driver.find_element(By.CSS_SELECTOR, ".add-to-cart").click()

        # Proceeds to checkout
        driver.find_element(By.CSS_SELECTOR, ".checkout-btn").click()

        # Completes purchase
        driver.find_element(By.ID, "email").send_keys("test@example.com")
        # ... complete checkout steps

        # Verifies success
        assert "Order Confirmed" in driver.page_source
```

6.2 Performance Testing

```python
```

```
# tests/performance/test_load.py
import asyncio
from locust import HttpUser, task, between

class ECommerceUser(HttpUser):
    wait_time = between(1, 3)

    @task(3)
    def view_products(self):
        self.client.get("/api/products")

    @task(2)
    def view_product_details(self):
        self.client.get("/api/products/1")

    @task(1)
    def create_order(self):
        self.client.post("/api/orders", json={
            "user_id": "123",
            "items": [{"product_id": "1", "quantity": 1}]
        })

    def on_start(self):
        # Login when user starts
        self.client.post("/api/auth/login", json={
            "email": "test@example.com",
            "password": "password"
        })
```

---

7. Deployment & DevOps

7.1 Containerization with Docker

```dockerfile
# Dockerfile
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements and install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Create non-root user
RUN useradd --create-home --shell /bin/bash app
```

````
USER app

# Expose port
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=30s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# Start application
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

### **7.2 Kubernetes Deployment**

```yaml
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
  labels:
    app: user-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: user-service
  template:
````

```
metadata:
  labels:
    app: user-service
spec:
  containers:
  - name: user-service
    image: myregistry/user-service:latest
    ports:
    - containerPort: 8000
    env:
    - name: DATABASE_URL
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: connection-string
    resources:
      requests:
        memory: "128Mi"
        cpu: "100m"
      limits:
        memory: "256Mi"
        cpu: "200m"
    livenessProbe:
      httpGet:
        path: /health
        port: 8000
      initialDelaySeconds: 30
      periodSeconds: 10
    readinessProbe:
```

```
        httpGet:
          path: /ready
          port: 8000
        initialDelaySeconds: 5
        periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: user-service
  ports:
  - port: 80
    targetPort: 8000
  type: ClusterIP
```

7.3 CI/CD Pipeline

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
```

```
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:13
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

    steps:
    - uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
```

```
      pip install -r requirements.txt
      pip install pytest pytest-asyncio

  - name: Run tests
    run: |
      pytest tests/ --cov=app --cov-report=xml

  - name: Upload coverage
    uses: codecov/codecov-action@v3
    with:
      file: ./coverage.xml

security-scan:
  runs-on: ubuntu-latest
  steps:
  - uses: actions/checkout@v3
  - name: Run security scan
    uses: aquasecurity/trivy-action@master
    with:
      scan-type: 'fs'
      scan-ref: '.'
      format: 'sarif'
      output: 'trivy-results.sarif'

deploy-staging:
  needs: [test, security-scan]
  if: github.ref == 'refs/heads/develop'
  runs-on: ubuntu-latest
  steps:
```

```
    - name: Deploy to staging
      run: |
        echo "Deploying to staging environment"
        # Deployment commands here

  deploy-production:
    needs: [test, security-scan]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
    - name: Deploy to production
      run: |
        echo "Deploying to production environment"
```

---

8. Monitoring & Observability

8.1 Logging Strategy

```python
# utils/logger.py
import logging
import json
from datetime import datetime
import sys

class JSONFormatter(logging.Formatter):
```

```
    def format(self, record):
        log_entry = {
            "timestamp": datetime.utcnow().isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "function": record.funcName,
            "line": record.lineno,
            "trace_id": getattr(record, 'trace_id', None),
            "user_id": getattr(record, 'user_id', None)
        }

        if record.exc_info:
            log_entry["exception"] = self.formatException(record.exc_info)

        return json.dumps(log_entry)

def setup_logging():
    logger = logging.getLogger()
    logger.setLevel(logging.INFO)

    # Console handler
    console_handler = logging.StreamHandler(sys.stdout)
    console_handler.setFormatter(JSONFormatter())
    logger.addHandler(console_handler)

    # File handler
    file_handler = logging.FileHandler('app.log')
```

```
    file_handler.setFormatter(JSONFormatter())
    logger.addHandler(file_handler)

# Structured logging in application
logger = logging.getLogger(__name__)

def process_order(order_id, user_id):
    extra = {'order_id': order_id, 'user_id': user_id, 'trace_id': '12345'}

    logger.info("Starting order processing", extra=extra)

    try:
        # Process order
        logger.info("Order processed successfully", extra=extra)
    except Exception as e:
        logger.error("Order processing failed", extra=extra, exc_info=True)
        raise
```

8.2 Metrics & Monitoring

```python
# utils/metrics.py
from prometheus_client import Counter, Histogram, Gauge, generate_latest
import time

# Define metrics
REQUEST_COUNT = Counter(
    'http_requests_total',
```

```
    'Total HTTP requests',
    ['method', 'endpoint', 'status']
)

REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'HTTP request duration',
    ['method', 'endpoint']
)

ACTIVE_USERS = Gauge(
    'active_users_total',
    'Number of active users'
)

# Middleware to collect metrics
async def metrics_middleware(request, call_next):
    start_time = time.time()

    try:
        response = await call_next(request)

        # Record metrics
        REQUEST_COUNT.labels(
            method=request.method,
            endpoint=request.url.path,
            status=response.status_code
        ).inc()
```

```
        return response

    finally:
        duration = time.time() - start_time
        REQUEST_DURATION.labels(
            method=request.method,
            endpoint=request.url.path
        ).observe(duration)

# Business metrics
def record_user_login(user_id):
    ACTIVE_USERS.inc()
    logger.info("User logged in", extra={'user_id': user_id})

def record_user_logout(user_id):
    ACTIVE_USERS.dec()
    logger.info("User logged out", extra={'user_id': user_id})
```

---

9. Security Best Practices

9.1 Authentication & Authorization

```python
# auth/jwt_handler.py
import jwt
from datetime import datetime, timedelta
```

```
from fastapi import HTTPException, status
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

class AuthHandler:
    def __init__(self):
        self.secret_key = "your-secret-key"  # From environment variable
        self.algorithm = "HS256"

    def get_password_hash(self, password: str) -> str:
        return pwd_context.hash(password)

    def verify_password(self, plain_password: str, hashed_password: str) -> bool:
        return pwd_context.verify(plain_password, hashed_password)

    def create_access_token(self, data: dict, expires_delta: timedelta = None):
        to_encode = data.copy()
        if expires_delta:
            expire = datetime.utcnow() + expires_delta
        else:
            expire = datetime.utcnow() + timedelta(minutes=15)

        to_encode.update({"exp": expire})
        encoded_jwt = jwt.encode(to_encode, self.secret_key, algorithm=self.algorithm)
        return encoded_jwt

    def verify_token(self, token: str):
        try:
```

```
            payload = jwt.decode(token, self.secret_key, algorithms=[self.algorithm])
            return payload
        except jwt.ExpiredSignatureError:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Token has expired"
            )
        except jwt.JWTError:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Invalid token"
            )

# Role-based access control
class RoleChecker:
    def __init__(self, allowed_roles: list):
        self.allowed_roles = allowed_roles

    def __call__(self, user: dict = Depends(get_current_user)):
        if user.get("role") not in self.allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Insufficient permissions"
            )
        return user

# Usage
admin_only = RoleChecker(["admin"])
manager_or_above = RoleChecker(["admin", "manager"])
```

```
@app.get("/admin/dashboard")
async def admin_dashboard(user: dict = Depends(admin_only)):
    return {"message": "Welcome to admin dashboard"}
```

9.2 Input Validation & Sanitization

```python
# utils/validation.py
from pydantic import BaseModel, validator, EmailStr
import html
import re

class UserRegistration(BaseModel):
    email: EmailStr
    password: str
    first_name: str
    last_name: str

    @validator('password')
    def validate_password_strength(cls, v):
        if len(v) < 8:
            raise ValueError('Password must be at least 8 characters')
        if not re.search(r'[A-Z]', v):
            raise ValueError('Password must contain at least one uppercase letter')
        if not re.search(r'[a-z]', v):
            raise ValueError('Password must contain at least one lowercase letter')
        if not re.search(r'\d', v):
```

```
            raise ValueError('Password must contain at least one digit')
        return v

    @validator('first_name', 'last_name')
    def sanitize_name(cls, v):
        # Remove HTML tags and escape special characters
        v = html.escape(v)
        # Remove any remaining suspicious characters
        v = re.sub(r'[<>{}]', '', v)
        return v.strip()

def sanitize_input(input_string: str) -> str:
    """Sanitize user input to prevent XSS and injection attacks"""
    if not input_string:
        return input_string

    # Remove HTML tags
    cleaned = re.sub(r'<[^>]*>', '', input_string)
    # Escape special characters
    cleaned = html.escape(cleaned)
    # Remove potential SQL injection patterns
    cleaned = re.sub(r'(\b(SELECT|INSERT|UPDATE|DELETE|DROP|UNION)\b)', '', cleaned,
flags=re.IGNORECASE)

    return cleaned
```

---

10. Learning Resources & Career Path

10.1 Essential Learning Platforms

System Design Fundamentals:
- **Chivankats Modern Institute** - Advanced System Architecture Curriculum
- **Educative.io** - Grokking the System Design Interview
- **System Design Primer** - GitHub repository with comprehensive guide

Cloud & DevOps:
- **AWS/Azure/GCP** - Official cloud certifications
- **Kubernetes.io** - Official documentation and tutorials
- **Docker** - Get Started guides

Advanced Topics:
- **Martin Fowler's Blog** - Enterprise patterns and practices
- **InfoQ** - Architecture and development news
- **The Twelve-Factor App** - Methodology for building SaaS applications

10.2 Recommended Study Path

```markdown
**Phase 1: Foundations (3-6 months)**
- Programming language mastery (Python/Java/Go)
- Database design and SQL
- Basic algorithms and data structures
- Version control with Git

**Phase 2: Core Development (6-12 months)**
```

- API design and development
- Microservices architecture
- Message queues and event-driven systems
- Testing strategies and TDD

Phase 3: Advanced Topics (12-24 months)

-